

Runtime FeedDownloader Pro

Podcast archiver

ADVANCED USER MANUAL

Version 1.6.0 · October 2026

“Digital content is fragile: a domain lapses, a hosting service shuts down, and years of publishing go with it. The only copy left is the one you saved.”

RUNTIME UTILITIES

ECOSYSTEM RUNTIME | DIGITAL CORE · LICENCE MIT

Chapter 1: Introduction and philosophy

1.1 What is Runtime FeedDownloader Pro?

It helps to start from the problem the software solves.

Thousands of podcast episodes are published and listened to every day. Over time a fair share of that material vanishes: the presenter stops paying for hosting, the distribution platform closes, the server holding the audio files is switched off. An episode you heard three years ago may now be gone for good — not because anyone deleted it deliberately, but because nobody kept a copy.

Runtime FeedDownloader Pro exists for that reason. It is neither a player nor an aggregator: it is an application for the systematic preservation of audio published through RSS feeds. It is meant for people who treat sound as a document, and therefore file it with the same care they would give a text or a photograph: archivists, newsrooms, radio stations, researchers, methodical listeners.

1.2 Who it is for

Four typical ways of using it.

- **The archivist** wants to pull down the whole catalogue of a long-running podcast before it disappears. What they need is a system that remembers what it already has, never duplicates anything and certifies the integrity of every file.
- **The radio producer** keeps the library on a shared NAS. They need a tool that works on network paths without seizing up, files things predictably and produces CSV inventories for the rest of the team.
- **The publisher** keeps a local copy of the network's podcasts, exports metadata to the editorial system and watches the state of the archive over time.
- **The listener** wants their podcasts on disk, in order, without depending on a connection or ending up with truncated files.

1.3 The database-first philosophy

What sets FeedDownloader Pro apart from a generic downloader is where the truth lives.

Almost every download tool reasons like this: look at the files on disk, compare them with the RSS feed, download whatever is missing. The weakness is obvious: the disk is not a reliable source. Files get moved, renamed, corrupted, deleted by mistake. Move the podcast folder from `C:\Podcasts` to `D:\Archive` and the tool loses every reference — then downloads the whole catalogue again.

FeedDownloader Pro keeps the truth elsewhere. At the centre of every operation is a SQLite database that records each episode it has met or downloaded: the source address, the file name, the folder it was saved in, the download date, the SHA-256 fingerprint of its content and its

audio metadata. The database is the software's memory, and it stays accurate even when the files move.

Five practical consequences follow.

1. **No duplicates.** Check the same feed ten times and the episodes already recorded never return to the ones to download.
2. **Every file knows where it is.** The program remembers each episode's folder: changing the download folder does not make the archive you have already built go "missing".
3. **Resilience to moves.** The archive can relocate to another disk or a NAS without losing its history.
4. **Persistent state.** If the program closes, or crashes, in the middle of a 300-episode batch, the history is intact when it reopens and the downloads still queued are offered for resuming.
5. **An audit trail.** Every downloaded file carries its date, its source address and a cryptographic fingerprint.

1.4 Three underlying technical choices

Beyond the database-first approach, the program rests on three decisions you can see directly in use.

Network resilience

Downloading hundreds of audio files from the internet fails often, and in many ways. Servers get overloaded, connections drop, transfers break off halfway. The software tackles this on four fronts.

- **Retries with growing waits.** When a download fails for a temporary reason, the program does not retry at once: it waits one second, then two, then four, then eight. There are five attempts in all. If the server explicitly asks for a pause (the `Retry-After` header), the program honours it, up to a maximum of sixty seconds.
- **Stall detection.** A stuck download is worse than a failed one: it hangs there and holds up the queue. The program watches the data flow and, if not a single byte arrives for sixty seconds, it closes the transfer and counts it as a failed attempt.
- **`.part` files with resume.** Every transfer writes to a temporary file with the `.part` extension. Only when the file is complete and its size matches what the server declared does it take its final name. If the transfer is interrupted, the `.part` stays on disk and the next attempt **resumes from where it stopped** instead of starting over.
- **Automatic pause when offline.** When the connection drops, the queue stops by itself and starts again when the network is back, instead of burning every episode's attempts in a few seconds.

Address safety

The program handles addresses that come from outside — from the feeds. A crafted address can point at resources on your local network (the router, the NAS, an internal server) and turn the application into a way of reaching them: the family of attacks known as SSRF, *server-side request forgery*.

Every address therefore goes through layered validation: a syntax check, HTTP and HTTPS only, blocking of known internal hostnames, blocking of private and reserved address ranges (RFC 1918, loopback, link-local) and a fresh check of the resolved IP address on every connection and

every redirect. It is automatic and invisible in use (chapter 10, section 10.8).

Network paths

The archive can live on a network drive. Handling SMB paths — the ones used by NAS units, Windows servers and shared folders — is a classic source of freezes in desktop applications: if the remote disk does not answer, the interface locks up until the operating system gives up. FeedDownloader Pro checks the path outside the graphical process and gives up on its own after eight seconds. The interface stays responsive regardless.

1.5 What changes in version 1.6.0

Coming from 1.5, you will find a reorganised program. In short:

- **A desktop interface.** The window is laid out in landscape, with side-by-side panels that each scroll on their own. The sidebar leads to the **Library**, **Downloads** and **Archive** views and, below them, to the feeds (chapter 3).
- **Downloading a period.** In every feed's view the filter strip is always visible and includes the **Period**, with ready-made ranges and free dates. The main button says exactly how many episodes it will take (chapter 4).
- **Automatic archiving per feed.** On the feeds you choose, new episodes found by the periodic checks download by themselves (chapter 5).

- **A queue that survives closing** and an **automatic pause when offline** (chapter 6).
- **Archive verification** in two modes, **Quick** and **Full**, with progress and the option to stop it (chapter 9).
- **Richer metadata next to the files**, with `podcast.json` and the cover for each podcast, and the new `{season}` and `{episode}` placeholders for file names (chapter 8).
- **From 1 to 10 parallel downloads**, **five attempts** per file and a **bandwidth limit shared** by all downloads (chapter 6).
- **Accessibility**: interface size from 80 to 180% and the **Reduce animations and effects** switch (chapter 10).

The release also fixes several 1.5 defects. The most important concerns ID3 tags: versions up to 1.5.x computed the SHA-256 fingerprint *before* writing the tags, so a full check then reported every tagged MP3 as altered. Section 9.5 explains how to put those files right.

1.6 How to read this manual

The manual covers the whole program, from installation to the least-used features. There is no need to read it in order: the chapters stand on their own.

To get your bearings, start with chapter 4, which follows a complete run from adding a feed to an archived file. If you already know the software, go straight to the chapter you need; if you are coming from 1.5, chapter 3 shows the new window layout.

Interface labels appear in **bold** and exactly as they read on screen. Navigation paths use an arrow: **Settings → Downloads → Change folder**.

Chapter 2 covers installation, first launch and updates.

Chapter 2: Installation and first launch

2.1 System requirements

Runtime FeedDownloader Pro is a desktop application built on Electron. It is self-contained: it does not need Node.js, .NET or Java installed, because everything it requires ships in the package.

Minimum requirements:

Operating system	Minimum version	Architecture
Windows	10 (build 1903) or Windows 11	64-bit (x64)
Linux	Ubuntu 22.04 LTS, Debian 11, Fedora 36 or equivalent	64-bit (x64)

Recommended hardware:

- **Memory:** 4 GB, better 8 GB for large archives with many parallel downloads
- **Disk:** 200 MB for the program, plus room for the audio archive
- **Screen:** the window does not go below 960 × 620 pixels; the full layout, with every panel side by side, is best from 1366 pixels wide upwards

- **Network:** broadband, at least 10 Mbps for parallel downloads to make sense

Note for Linux. The software ships as an `.AppImage` (self-contained, runs on any recent distribution without installing) and as a `.deb` for Debian- and Ubuntu-based distributions.

Note on macOS. There are no official macOS builds and none are planned. The source code can be compiled on a Mac, but building it is up to whoever does so, and it is not supported.

2.2 Installing on Windows

1. Download `Runtime-FeedDownloader-Pro-1.6.0.exe` from the project's releases page.
2. Double-click the downloaded file.
3. If the **Windows protected your PC** warning (SmartScreen) appears, click **More info** and then **Run anyway**. The warning is normal for programs distributed outside the Microsoft Store and without a commercial signature: it is about the file's reputation, not its contents.
4. Choose the installation folder and click **Install**.
5. When it finishes, you will find the shortcut on the desktop and in the Start menu.

Where the files go. Installation is per user and needs no administrator rights: the program goes into `C:\Users\[YourName]\AppData\Local\Programs\Runtime FeedDownloader Pro\` unless you choose otherwise during setup. The database lives elsewhere, in

`C:\Users\[YourName]\AppData\Roaming\Runtime FeedDownloader Pro\`, so uninstalling the program leaves the archive untouched.

2.3 Installing on Linux

1. Download `Runtime-FeedDownloader-Pro-1.6.0.AppImage`, or the `.deb` package for Debian and Ubuntu.
2. Make the AppImage executable, in either of two ways:
   - from the desktop: right-click the file → Properties → Permissions → tick "allow executing as a program";
   - from a terminal: `chmod +x Runtime-FeedDownloader-Pro-1.6.0.AppImage`.
3. Launch it with a double-click or from a terminal: `./Runtime-FeedDownloader-Pro-1.6.0.AppImage`.

For the Debian package: `sudo dpkg -i Runtime-FeedDownloader-Pro-1.6.0.deb`, or your distribution's graphical package manager.

Desktop integration. To get an icon in the applications menu you can use **AppImageLauncher**, available in most distributions' repositories, which registers AppImages with the system.

Note for sandboxed environments. Under **Flatpak** or other environments that restrict file-system access, the program may not see SMB network paths. In that case, check that the share is mounted and reachable from the file manager before starting the application.

Note for GNOME. The notification-area icon needs the AppIndicator extension. Without a notification area, closing the window also quits the program (section 3.13).

2.4 First launch

The entry screen

When the program opens, the entry screen appears: the program's mark inside an animated orbit, the name and a line summing up what it is for. The animation lasts under four seconds and then stops; with **Reduce animations and effects** turned on (chapter 10), the screen appears already composed.

On the same screen you will find:

- the **Interface language** choice, **Italiano** or **English**: on first launch the program proposes the operating system's language;
- a status line about updates (*Checking for updates…*, or the version available);
- the **Enter FeedDownloader** button, which takes you into the interface; it already has the focus, so pressing `Enter` is enough;
- three links: **Guide** (the built-in documentation), **Manual** (this manual as a PDF) and **Support the project**.

The screen returns every time the program opens.

The empty library

Once in, the software is ready to use. There is no setup, no account, no licence key. With the library empty, the **Library** view shows a welcome panel with two buttons, **Add feed** and **Import from OPML**, and the three steps to get started: add a feed, choose the folder, archive.

The default download folder. Until you choose another, files go to `Documents\FeedDownloader\downloads` (on Linux `~/Documents/FeedDownloader/downloads` or its localised equivalent). You change it from **Settings → Downloads → Change folder** (chapter 4, section 4.3).

Files created on first launch. A single file appears in the user data folder:

- `feeddownloader.sqlite`, the database. It holds the download history, the archive with its fingerprints, the feed list, the saved queue and your preferences (download folder, parallel downloads, bandwidth and size limits, check interval, interface size). **Do not delete this file.**

The interface language is the exception: the program remembers it alongside the display preferences, not in the database, and you change it from the entry screen or from **Settings → General**.

2.5 Updates

Updates always need your explicit consent. The path has three steps.

1. At launch, and on request from **Settings → About → Check for updates**, the program checks whether a newer version exists. If it

finds one, the **Version X.Y.Z available** notice appears in the top bar, together with a system notification. The notice stays until you act; the × (**Hide notice**) removes it until the next restart or the next check.

2. The download starts **only** when you press **Download** on the notice or in Settings. While it downloads, the notice reads **Downloading update N%**. Nothing is downloaded on the program's own initiative.
3. Once the download completes, the notice becomes **Update ready**: pressing **Restart and install** closes the program, applies the update and starts it again. Installation does not happen by itself either, and it is not triggered when the application closes.

The update replaces the program's files and leaves the archive alone. On the first launch of the new version, the **What's new** window summarises what has changed; you can reopen it at any time from **Settings → About → What's new**.

Moving from 1.5 to 1.6. On first opening, 1.6.0 adds the new columns and tables to the database (each episode's folder, automatic archiving, the saved queue) and completes the existing data: the episodes already archived are linked to the download folder in use at that moment. No data is removed.

Tip. Before a major version jump like this one, copy `feeddownloader.sqlite` somewhere safe. If you used ID3 tagging in earlier versions, section 9.5 explains how to refresh the fingerprints of the files already downloaded.

Chapter 3 describes the interface view by view.

Chapter 3: A tour of the interface

3.1 How the window is laid out

FeedDownloader Pro is a desktop application, designed for a landscape window. Panels sit **side by side** and each one scrolls on its own: the episode list scrolls without dragging the filters along, the download queue without moving the results.

The window has three areas.

- **The top bar.** It shows the state of the download queue and holds the global commands: **Check feeds**, search (Ctrl+K), the **Guide** and **Settings**. When an update exists, its notice appears here too (chapter 2, section 2.5).
- **The sidebar**, on the left. At the top it leads to the three cross-cutting views, **Library**, **Downloads** and **Archive**; below them it lists the feeds in the library; at the bottom it has the **Add feed** and **Import from OPML** buttons.
- **The main area**, in the centre and on the right. It shows the chosen view: the Library, a feed, Downloads or the Archive.

The more self-contained tasks, such as adding a feed, the settings or the archive verification, open over the interface as dialogs (section 3.10).

3.2 The top bar

From left to right:

- **The mark**, with the program's name.
- **The update notice**, only when an update is available, downloading or ready.
- **The queue status.** A pill says in words what is happening: **Queue empty** (with a summary of the last run, for instance *last run: 12 downloaded, 1 failed*), **3 downloading** (with progress, for instance *47 of 312*), **Queue paused**, or **Offline**. A coloured dot goes with the status: green while downloading, yellow when paused, red with no network. Clicking the pill opens the **Downloads** view.
- **Check feeds.** Reads every feed in the library again right away, looking for new episodes, and starts automatic archiving where it is on (chapter 5, section 5.9). While it works the label reads *Checking 3/7*. Shortcut: **Ctrl+S**.
- **The magnifying glass**: opens the command palette (section 3.11). Shortcut: **Ctrl+K**.
- **The book**: opens the **Guide**, the built-in documentation.
- **The gear**: opens **Settings**, even while a download is running (chapter 10). Shortcut: **Ctrl+,**.

In windows narrower than 1024 pixels, **Check feeds** shows as an icon only and the queue status detail is hidden.

3.3 The sidebar

The three views

At the top are three entries, always visible.

- **Library** (Ctrl+1): the dashboard of every feed (section 3.4). Next to it is the total of episodes still to download, for instance 42 *to download*.
- **Downloads** (Ctrl+2): the queue, the interrupted, the failed and the completed downloads (section 3.8). Next to it is the number of queued downloads or, with the queue empty, the number of failed ones (2 *failed*).
- **Archive** (Ctrl+3): every downloaded episode (section 3.9), with their number.

The entry of the open view is marked by a violet bar on its left edge.

The feed list

Below the views is the list of feeds, with their number and the **Sort A to Z** button: on, it puts the feeds in alphabetical order; off, it restores the order they were added in. The choice is remembered. With at least one feed in the library the **Search feeds…** field appears too; it filters by title and by address as you type.

Each row shows the cover, the title declared in the feed and a status line:

- **N to download**, when there are episodes not archived yet;
- **All archived**, when nothing is missing;
- **Not checked yet**, for a feed the program has not read yet, for instance one just imported from an OPML file.

Feeds with automatic archiving switched on also carry the green **auto** mark.

Clicking a row opens the feed in the main area; while the program reads it, a spinner turns over the cover. A feed that is already open comes back at once, without reading it again.

At the bottom

Add feed (Ctrl+N) opens the add dialog (section 3.10). **Import from OPML** imports a feed list from another application (chapter 5, section 5.5).

Removing a feed is not done from the sidebar: the **Remove** command lives in the feed's header (section 3.5), away from a stray click.

3.4 The Library view

This is the opening view. With the library empty it shows the welcome panel (chapter 2, section 2.4); otherwise it presents the dashboard of the whole library.

The Library header

The title **Your library**, the **Check feeds** and **Add feed** buttons, and three figures: **Feeds**, **To download** and **Archived**. During a feed check, progress appears under the title (*Checking feeds: 3 of 7…*).

The feed table

One row per feed, with these columns:

Column	Contents
Podcast	Cover, title and, when on, the **auto** mark of automatic archiving
Episodes	How many episodes the feed lists
Archived	How many of the feed's episodes are in the archive
To download	How many are missing; **complete** when nothing is, a dash if the feed has not been checked yet
Space	The space taken by the feed's archived files
Last checked	Date and time of the last read of the feed; **never** if it has not happened yet

Clicking a column header sorts the table by that value; a second click reverses the order. Clicking a row opens the feed.

The panels beside the table

In wide windows (from about 1320 pixels), next to the table there is a column with three panels:

- **Download queue**: the state of the queue and the progress of the run, with **Open Downloads**;
- **Archive**: how many episodes are archived and how much space they take, with **Open the Archive** and **Import from OPML**;
- **Keyboard shortcuts**: a reminder of the main key combinations (section 3.12).

In narrower windows the column disappears and the table takes all the room; the **Episodes** column disappears too.

3.5 The Feed view

It opens when you click a feed. It is laid out in two horizontal strips, the header and the filters, above a working area split in two: the episode list on the left and a panel on the right.

The feed header

- **Cover and title** of the podcast, with the **Automatic archiving** badge when it is on.
- **Checked** followed by the date and time of the last read, and the path of the folder this podcast's files go to.
- **The feed commands**:
  - **Refresh**: reads the feed from the server again, ignoring the copy read a few minutes earlier, and tells you how many new episodes it found. It downloads nothing, except for feeds with automatic archiving (chapter 5, section 5.10);
  - **Open folder**: opens the podcast's folder in the file manager;
  - **Export M3U playlist**: creates a playlist of the podcast's archived files (chapter 5, section 5.7);
  - **Copy address**: copies the feed address to the clipboard;
  - **Remove**: takes the feed out of the library, after a confirmation (chapter 5, section 5.4).

In windows at least 1600 pixels wide the commands have written labels; narrower than that, only the icon remains, which shows its name when you hover over it.

- **The figures**: **Episodes**, **Archived**, **To download** and **Space used**. Below 1280 pixels only the space used remains, because the other three can already be read in the status filters.

The filter strip

Always visible, on one line. Filters combine with each other.

- **Search** (*Search titles and descriptions…*): filters by keywords in the title and description. AND logic applies: every word you type must appear. **Ctrl+F** puts the cursor here.
- **Status**: three buttons, **All**, **To download** and **Archived**, each with its count. **To download** also leaves out episodes already queued.
- **Period**: a menu of ready-made ranges, **Whole feed**, **Last 30 days**, **Last 90 days**, **Last 12 months**, **This year**, **Last year**, and **Chosen dates**.
- **From** and **To**: the publication dates, both included. Choosing a ready-made range fills in these two fields; editing them by hand switches the menu to **Chosen dates**. You can fill in just one end.
- **Duration**: minimum and maximum in minutes. Episodes that do not declare a duration are not excluded.
- **Order**: **Feed order** (default), **Newest first**, **Oldest first**, **Title (A–Z)**, **Longest first**, **Shortest first**.
- **Clear filters**: appears when at least one filter is active and resets them all.

Opening another feed resets the filters.

The action bar

Above the list, a line says what the main button will do:

- the summary, for instance *120 of 312 episodes shown · 87 to download*, followed by the active period (*published from 1 January 2025 to 30 June 2025*) and the number of selected episodes;
- with a selection active, **Deselect** and **Download the N selected**;
- the **main button**, which says exactly how many episodes it will take: **Download the N missing** with no filters, **Download the N filtered** with at least one filter active. It counts only the visible episodes that are neither archived nor already queued.

The button stays disabled with no network or when there is nothing to download.

The episode list

One row per episode, in three columns:

- **Episode**: the title and, below it, publication date, duration, numbering (for instance `S2 E7`, when the feed declares it) and declared size;
- **Status**: the episode's status (section 3.6), with the progress bar while it downloads;
- **the actions**, always visible: the download icon for an episode to download, the × to cancel a queued episode, the folder icon to show an archived episode in the file manager.

Clicking the title opens the episode details (section 3.7).

The list draws only the rows visible on screen, so it stays smooth even with thousands of episodes.

Selection

Each row has its own checkbox.

- Clicking the checkbox adds or removes that episode.
- **Shift** + click on a checkbox selects the range from the last episode chosen to the one clicked.
- The checkbox in the list header selects, or deselects, every episode shown.
- **Ctrl+A**, outside text fields, selects every visible episode.

The selection clears once the download starts and when another feed is opened.

The right-hand panel

In windows at least 1280 pixels wide there is a fixed panel to the right of the list. When no episode is open, it contains:

- the **Automatic archiving** switch, with its description (chapter 5, section 5.10);
- **What will be downloaded**: the main button's plan, with the number of **Episodes**, the **Estimated space** and the active period. The text says whether the plan covers every episode not archived, the filtered ones or the selected ones, and the estimate updates as the filters change;
- **The podcast**: the description published in the feed and its address.

When you open an episode, the panel shows its details; the **Back to the feed card** arrow returns to the previous view.

Below 1280 pixels there is no panel: the **Automatic archiving** switch moves into the action bar, and the episode details open in a side window.

3.6 Episode states

Every episode carries a status, written out in words. Reading it is the quickest way to see where the archive stands.

Status	Where you see it	Meaning
To download	Status column	The episode is in the feed and not in the archive.
Queued	Status column, with a spinner	Waiting for its turn.
Progress	Bar with percentage, speed and time left	Transfer under way.
Paused	Still yellow bar	Suspended; resumes from where it stopped.
Failed	Status column, in red; reason in the Downloads view	The download failed after the automatic attempts, or with a permanent error.
Archived	Status column, in green	The file is recorded in the archive, from this session or an earlier one.

The **Archived** status follows directly from the database-first approach. Reopen a feed you have already worked on and almost every episode shows as archived: the program knows it has them.

If the server does not declare the file size, the progress bar becomes indeterminate and the bytes received scroll in place of the percentage.

3.7 Episode details

They open when you click the title. In wide windows they take the right-hand panel; in narrow ones they appear in a side window.

From the top, they contain:

- **The badges**: **Archived** or **To download**, and the episode type when the feed declares it as something other than a regular episode (for instance `trailer` or `bonus`).
- **The feed data**: **Published**, **Duration**, **Numbering** (season and episode), **Declared size** and **Format** (the declared MIME type).
- **In the archive**, for archived episodes only: **Downloaded on**, **Size on disk**, **Quality** (bitrate and sample rate), **File** (folder, podcast and file name) and **SHA-256 fingerprint (recorded at download)**.
- **Episode notes**: the descriptive text published in the feed, stripped of HTML.
- **Addresses**: the episode's web page with **Copy page** and the audio file with **Copy audio**.

At the bottom, the actions:

- **Download**, for an episode to download; it becomes **Queued** while it waits.
- **Download again**, for an archived episode. It downloads a new copy: if the previous file is still in the folder, the new copy gets the `_2` suffix and the archive switches to it. The previous file is not deleted.
- **Show in folder**: opens the file manager on the archived file.
- **Mark as not downloaded**: takes the episode out of the archive, after a confirmation, and puts it back among the ones to download. The file on disk is not deleted.

3.8 The Downloads view

This is where you watch and control everything that is downloading. Open it from the sidebar, with **Ctrl+2** or by clicking the queue status in the top bar. Closing the window or switching views stops nothing.

The Downloads header

The title **Downloads** and a sentence about the state (the queue works in the background, is paused, is waiting for the network, or is empty). Then the commands that apply to the whole queue:

- **Pause**: suspends every transfer, keeping the partial files; it becomes **Resume** while the queue is paused;
- **Stop and clear the queue**: stops everything, after a confirmation (section 6.6).

Last, two figures: **Running** and **Waiting**.

Below the header, during a run, a bar shows overall progress: *47 of 312 finished*, with the percentage.

The queue

The left pane lists the queued downloads: active ones first, then waiting, then paused. Each row shows the episode title, the podcast and the state (*downloading*, *waiting*, *paused*); active or suspended downloads also show the bar with percentage, speed and time left. On the right of each row are pause (or resume) for that download alone and the × to cancel it.

Interrupted, failed and completed

In windows at least 1280 pixels wide, beside the queue there is a column with three sections.

- **Downloads interrupted when the app last closed.** It appears only when, at launch, downloads were left queued from the previous session — after closing with the batch still running, or a crash, for instance. It says how many there are and from which podcasts, and offers **Resume** or **Discard** (chapter 6, section 6.8).
- **Failed.** The downloads that failed in this session, each with its reason in plain words and the technical code in brackets, for instance *The file no longer exists on the server* (404). (EPISODE_NOT_FOUND). Each row has **Retry** and the × to remove it from the list; at the top are **Clear list** and **Retry all (N)**.
- **Completed.** The episodes archived in this session, with size and time, and the total downloaded (*This session · 2.4 GB downloaded*). Each row has the icon to show the file in its folder; **Clear list** removes the rows without touching the files.

Below 1280 pixels the three lists become tabs: **Queue**, **Failed** and **Completed**, each with its count. The interrupted-downloads panel stays at the top, above the tabs.

3.9 The Archive view

Reach it from the sidebar or with **Ctrl+3**. Where the Feed view shows one podcast at a time, here every downloaded episode, from any podcast, appears in a single table.

The Archive header

The title **Archive** and the range covered (*Episodes published from … to …*). Then two commands:

- **Verify archive**: opens the check of file presence and integrity (chapter 9, section 9.5);
- **Export CSV**: saves the full inventory (chapter 9, section 9.7).

Last, three figures: **Episodes**, **Podcasts** and **Space used**.

The filter column

In windows at least 1360 pixels wide, to the left of the table there is a column with:

- **search** (*Title, podcast, file name or date…*): searches the episode title, the podcast name, the file name and the dates, both as shown on screen and in ISO form (`2026-08-18`). **Ctrl+F** puts the cursor here;

- the **Publication period**: **Whole archive**, the same ready-made ranges as the Feed view, **Chosen dates**, and the **From** and **To** fields;
- the list of **Podcasts**: **All podcasts** and then each podcast in alphabetical order, with the number of its archived episodes. Clicking a podcast filters the table; hovering shows the space it takes too.

Below 1360 pixels the same filters sit in a strip above the table, and podcasts are chosen from a drop-down menu.

The table

Above the table, a line sums up the selection (*240 of 1,180 episodes · 11.2 GB*) and offers **Clear filters** when a filter is active. The columns are **Podcast**, **Episode**, **Published**, **Downloaded**, **Size** and **Quality** (the last two only in wide windows). Clicking a header sorts by that column; by default the table is sorted by download date, newest first. The folder icon at the end of the row shows the file in the file manager.

The table updates itself after every completed download.

3.10 Dialogs

Some features open in a window over the interface. Close them with **Esc**, with the × or with their close button.

- **Add a feed** (Ctrl+N): the **Feed address** field, the **Analyse** button, the podcast preview and the **Add to library** button (chapter 4, section 4.4).
- **Settings** (Ctrl+,): six vertical tabs, described in chapter 10.

- **Archive verification**: the **Quick** and **Full** checks and the actions on missing or altered files (chapter 9, section 9.5).
- **Guide**: the built-in documentation, with the link to this manual as a PDF.
- **What's new**: the release notes of the installed version. It appears by itself on the first launch after an update; the **Got it** button closes it.

Confirmation requests (removing a feed, downloading several episodes, resetting the history) use smaller windows, whose main button says exactly what it will do.

3.11 The command palette (Ctrl+K)

The palette reaches the main features without the mouse. Open it with **Ctrl+K**, or the magnifying glass in the top bar, from anywhere in the program, even during a download. The window's title is *Search actions, feeds and episodes*.

Type to filter, move with the ↑↓ **arrows** (**Home** and **End** jump to the first and last result), confirm with **Enter**, leave with **Esc** without running anything.

Results are grouped.

- **Actions**: **Add a feed**, **Check every feed**, **Go to Library**, **Go to Downloads**, **Go to Archive**, **Verify archive**, **Import from OPML**, **Settings** and **Guide**. Each shows its shortcut, where one exists.
- **Feeds**: with the field empty it shows the first feeds in the library; as you type it filters them by title. Choosing one opens it.

- **Episodes of [podcast]**: as you type, the palette also searches the episode titles of the open feed. Choosing one takes you back to the feed view with the search already filled in with that title.

3.12 Keyboard shortcuts

Combination	Effect
Ctrl+K	Opens the command palette
Ctrl+N	Opens **Add a feed**
Ctrl+S	**Check feeds** (outside text fields)
Ctrl+F	Puts the cursor in the open view's search (Feed or Archive)
Ctrl+A	Selects every visible episode (Feed view, outside text fields)
Ctrl+1	**Library** view
Ctrl+2	**Downloads** view
Ctrl+3	**Archive** view
Ctrl + and **Ctrl –**	Enlarges or shrinks the interface
Ctrl 0	Returns the interface to 100%
Ctrl+,	Opens **Settings**

Inside a text field **Ctrl+A** keeps its usual meaning (select all the text). Right-clicking in a text field opens the menu with cut, copy, paste and select all.

Every command can also be reached from the keyboard, with **Tab** and **Shift+Tab**, and the focus is always visible.

3.13 The notification-area icon

Closing the window with the × does not quit the program: it shrinks to the notification area, near the clock. This is deliberate, because downloads and the automatic check for new episodes carry on with the window closed.

Clicking the icon shows or hides the window. The right-click menu has two entries: **Show** brings the window to the front, **Quit** closes the program and stops active downloads. The downloads left in the queue are not lost: when you reopen the program, the Downloads view offers to resume them (chapter 6, section 6.8).

If the system offers no notification area (for instance GNOME without the AppIndicator extension), closing the window quits the program.

In practice: for a large archive, start the batch, close the window and let the machine work.

3.14 Narrow windows and an enlarged interface

The layout stays landscape even when space shrinks: side panels give way to tabs or windows, but the page never stacks into a single column.

Window width	What changes
Below 1600 pixels	Feed commands show as icons
Below 1360 pixels	Archive: the filters move into a strip above the table; Size and Quality disappear
Below 1320 pixels	Library: the right-hand column and the Episodes column disappear
Below 1280 pixels	Feed: details open in a side window and automatic archiving moves into the action bar; Downloads: the lists become tabs
Below 1024 pixels	**Check feeds** shows as an icon

The thresholds are in interface pixels: enlarge the interface, say to 140%, and the same screen offers less room, so the thresholds kick in sooner. The window does not go below 960 × 620 pixels; when the height is not enough, the main area scrolls.

Chapter 4 walks through a complete run, from adding a feed to the archived file.

Chapter 4: Your first archive, step by step

4.1 The route

This chapter follows a whole route: from a podcast's address to a folder of tidy audio files on disk. The scenario is the most common one: downloading a programme's catalogue for the first time, all of it or just a period.

It is worth reading once from start to finish. Once you have the hang of it, starting a new archive takes less than a minute.

4.2 Finding the feed address

You start from the podcast's RSS address. An RSS feed is an XML text file that the publisher puts online to list the available episodes: every podcast has one, even when it does not advertise it.

Where to look:

- **On the podcast's website.** An orange icon with waves, or the words RSS, Feed, Subscribe, Podcast Feed. Clicking usually opens the XML file in the browser: the address in the bar is the one you need.

- **In a listening app.** Pocket Casts, Apple Podcasts and the like often show the RSS link in the programme's information, sometimes under Share.
- **On the hosting platform.** Spreaker, Podbean, Buzzsprout and similar services publish the feed address on the podcast page or in the producer's dashboard.
- **With a search engine.** Searching for `[podcast name] RSS feed` often finds it first time.

A valid RSS address usually ends in `.xml` or `.rss`, or has `feed`, `rss` or `podcast` in its path. For example: `https://www.example.com/feed.xml`, `https://feeds.spreaker.com/podcast/12345`, `https://anchor.fm/s/abc123/podcast/rss`.

4.3 Choosing the download folder

Before adding the feed, decide where the files will go. Inside the download folder the program creates a subfolder for each podcast by itself, and saves the files there with the names set by the template (chapter 8):

```
D:\Podcast Archive\
    ├── My Podcast\
    ├── Tech Podcast\
    └── Radio Talk Show\
```

Without an explicit choice, the folder is `Documents\FeedDownloader\downloads`. To change it:

1. Open **Settings** → **Downloads** (the gear at the top right, or **Ctrl+,**).

2. Press **Change folder** in the **Download folder** section.
3. Choose the folder and confirm. The program checks it can write there and confirms with *Download folder updated.*

If the archive already holds episodes, the program asks what to do with them: move them to the new folder, or leave them where they are and use the new one only for future downloads (chapter 9, section 9.8). For a first archive the question does not appear.

If the archive lives on a NAS or a network drive, read chapter 7 before going on: those paths need a little extra care.

4.4 Adding the feed

1. Press **Add feed** at the bottom of the sidebar, or **Ctrl+N**.
2. Paste the address into the **Feed address** field. The `https://` prefix may be left out, and addresses starting with `feed:` or `podcast:` are converted automatically.
3. Press **Analyse**, or just `Enter`. For a 200- or 300-episode podcast a few seconds are enough; catalogues of over a thousand, or split across many pages, can take fifteen to thirty.
4. The preview appears: cover, title, number of episodes and the range covered (*from … to …*), with the start of the description. This is the moment to check it is the right podcast.
5. If you like, switch on **Automatic archiving** here: new episodes found by the periodic checks will download without asking (chapter 5, section 5.10).
6. Press **Add to library**. The feed joins the sidebar and its view opens.

If the feed is already in the library, the preview says so and the button becomes **Open feed**.

Two shortcuts. You can drag an address from the browser straight onto the program's window: **Add a feed** opens with the analysis already running. And if you already have a podcast list in another application, you can bring it in with **Import from OPML** (chapter 5, section 5.5).

If the analysis fails, the reason appears under the field, for instance *The address leads to a web page, not to an RSS feed.* or *The server is not answering. Check the connection.* Chapter 11 lists every message and what it means.

4.5 Reading the feed view

With the feed open, look at three things.

- **The figures** in the header: **Episodes**, **Archived**, **To download** and **Space used**. A programme that has been running for a few years easily has a few hundred episodes.
- **Episodes already archived**, if the feed was worked on in the past: the database knows them and will not download them again. The **Archived** filter shows them all together.
- **Episodes with no duration or size**: the feed simply does not declare them. They download just the same, with no consequence; the space estimate uses an average value for them.

4.6 Choosing what to download

The main button, above the list, always takes the episodes **shown** that are neither archived nor already queued, and says so on its label. There are three ways to use it.

The whole catalogue

With no filters active the button reads **Download the N missing**: it takes every episode not yet archived.

A period

To download “from day X to day Y”:

1. In the filter strip, open the **Period** menu and choose a ready-made range (**Last 30 days**, **Last 90 days**, **Last 12 months**, **This year**, **Last year**), or fill in the **From** and **To** dates.
2. If you like, combine other filters: the **To download** status, a minimum duration, a word in the title.
3. The button becomes **Download the N filtered**, and the **What will be downloaded** panel on the right shows the number of episodes, the estimated space and the period.

A selection

Tick the checkboxes of the episodes you want; **Shift** + click on a checkbox takes a range, **Ctrl+A** every visible one. **Download the N selected** appears.

For a single episode, the download icon on its row is enough, or **Download** in the details.

4.7 Confirming and starting

For more than one episode the program asks for confirmation. The *Download N episodes?* window shows:

- how many episodes go into the queue and from which podcast;
- the **Period**, if a date filter is active;
- the **Estimated space** and the **Free space** on the destination disk;
- the **Folder** the files will go to.

If the free space may not be enough (less than one and a fifth times the estimate), the window says so in yellow. If less than 200 MB is left on the disk, the program starts nothing and says so: *Almost no space left (… free): downloads not started.*

Pressing **Queue** starts the downloads, as many at a time as set in **Settings → Downloads → Parallel downloads**: three unless you change it (chapter 6).

The space estimate uses the size declared by the feed; where that is missing, it works it out from the duration (at 192 kbps), and without that either it counts 45 minutes per episode.

4.8 Following progress

The top bar always tells you how many downloads are running and how far the run has got (*3 downloading · 47 of 312*). Rows in the episode list show their own bar, with percentage, speed and time left.

For the full picture there is the **Downloads** view (**Ctrl+2**, or click the queue status): the queue with its pause and cancel controls, the failed downloads with their reason, this session's completed ones.

You do not need to keep the window open: close it and the program stays in the notification area and keeps working. When the batch ends, a system notification arrives (*Download complete: 312 files downloaded.*).

During the batch the software handles retries after network errors, stalls and the check of every completed file. If the connection drops, the queue pauses by itself and starts again when the network is back. If the computer goes to sleep, on waking the stall watch closes the frozen transfers and the retry cycle usually recovers; episodes that had already used up their five attempts end up among the failed ones, from where **Retry all** puts them back in the queue.

4.9 Checking the result

When the queue empties, three things remain.

Check the failed downloads. In the **Downloads** view, the **Failed** section lists the missed episodes with their reason. The most common is *The file no longer exists on the server (404).*, a file removed before the program could fetch it. **Retry all (N)** puts them back in the queue in one go.

Look at the disk. Open folder, in the feed header, opens the podcast's folder. The files carry the names set by the template (chapter 8). If you find `.part` files, those transfers were interrupted or paused: queue the same episodes again and they resume where they stopped. Orphaned ones are removed from **Settings → Archive → Clean temporary files**.

Export the inventory. In the **Archive** view, **Export CSV** produces the full list of archived episodes with SHA-256 fingerprint, sizes, metadata and folder (chapter 9).

4.10 Keeping the archive up to date

This is where the database-first approach pays off: the program always knows what it owns, and the same episode is never downloaded twice.

- **By hand. Check feeds** (top bar, or **Ctrl+S**) reads the whole library again. Feeds with new episodes show *N to download* in the sidebar and in the Library; open the feed and **Download the N missing** takes only those. For a single feed there is **Refresh** in its header.
- **By itself.** The program checks every feed at launch and then every six hours, even with the window closed (change the interval in **Settings → General**). When it finds new episodes it sends a system notification.
- **Hands-off.** With **Automatic archiving** on for a feed, new episodes found by the checks queue themselves (chapter 5, section 5.10). It is the simplest way to follow a podcast over time.

Chapter 5 goes into managing feeds, OPML files and checks in detail.

Chapter 5: Managing feeds

5.1 What an RSS feed is

An RSS feed is an XML document a podcast publishes so that programs can read its list of episodes. When a new episode comes out, the publisher adds an entry to that document; readers fetch it again from time to time and notice the change.

For FeedDownloader Pro the feed is the source of everything: it holds the list of episodes, the addresses of the audio files, the metadata (title, date, duration, numbering, description, cover) and general information about the programme.

You do not need to know its internal structure to use the software. It does help to understand why data is sometimes missing from the list: almost always, the feed does not declare it.

5.2 Well-made feeds and problem feeds

Standards compliance, in the podcast world, is more an aspiration than a rule.

Well-formed feed. It follows RSS 2.0 or Atom, declares title, link, date and audio address with its MIME type, and perhaps adds iTunes or

Podcast Index tags for duration, cover, season and episode. The program reads it without trouble.

Incomplete feed. Optional fields such as duration, size, numbering or episode cover are missing. The files download all the same, but some information in the list will stay blank.

Feed with unreachable audio. The document reads fine, but the file addresses point to resources that have gone. It happens a lot with abandoned or migrated programmes. The episodes end up among the failed ones after the download attempt.

Password-protected feeds. Some private podcasts ask for HTTP Basic credentials. You can put them straight into the address: `https://user:password@www.example.com/feed.xml`. The credentials then stay written in the address saved in the library and in OPML exports.

5.3 What happens when you press Analyse

In order:

1. **Address check.** Correct syntax and passing the checks against SSRF attacks (chapter 10, section 10.8).
2. **Request to the server.** The program introduces itself with a recognisable name (`RuntimeFeedDownloader`) and gives up after fifteen seconds without an answer. The same address cannot be requested twice less than three seconds apart.
3. **Reading the document.** It reads RSS or Atom and tolerates the commonest deviations: undeclared encoding, missing tags, made-up

namespaces. If the address leads to a web page rather than a feed, it says so.

4. **Further pages.** If the feed is split into linked pages, it follows them and rebuilds the catalogue (section 5.8).
5. **Preview.** The **Add a feed** window shows cover, title, number of episodes and the range covered.
6. **Joining the library.** With **Add to library** the feed joins the sidebar. At that moment the program records the unique identifiers of the episodes present, the GUIDs: they become the reference for recognising, in later checks, which episodes are new.

To recognise episodes already archived, the program uses the GUID; when the feed does not declare one, the audio file's address stands in for it.

The feed it has read stays in memory for five minutes: reopening it within that time does not send a new request to the server. **Refresh**, in the feed header, bypasses that copy and reads it again at once.

5.4 The feed library

Feeds you add stay saved: there is no need to paste addresses again at every launch. The library shows in two places.

- **In the sidebar**, with cover, title and a status line: **N to download**, **All archived** or **Not checked yet**, and the **auto** mark for automatic archiving. Above the list are **Search feeds…** and **Sort A to Z** (chapter 3, section 3.3).

- **In the Library view**, as a sortable table with episodes, archived, to download, space used and date of the last check (chapter 3, section 3.4).

The count of episodes to download comes from comparing the feed's episodes with those recorded in the archive.

Removing a feed

1. Open the feed.
2. Press **Remove** in the header (the bin icon in narrower windows).
3. Confirm with **Remove feed**.

Removal deletes the feed from the library but **does not touch the audio files already downloaded** or their records: the episodes stay in the Archive view.

On privacy. The library lives only in the local database. The program sends your feeds to no one.

5.5 Importing feeds from an OPML file

OPML is the format podcast applications use to exchange subscription lists. A library kept in Pocket Casts, Overcast, AntennaPod or similar can be exported and brought in here.

1. Press **Import from OPML**: you will find it at the bottom of the sidebar, in the Library view, in the **Add a feed** window, in **Settings → Archive** and in the command palette.
2. Choose the `.opml` (or `.xml`) file.

3. The program adds the feeds it finds to the library and confirms it (*12 new feeds imported.*).

Straight afterwards a check of every feed starts, fetching real titles, covers and counts in place of the names written in the OPML file.

Worth knowing:

- importing **does not change feeds already in the library**: if an address is already there, it stays as it was (*No new feeds: they were all in the library already.*);
- only HTTP and HTTPS addresses that pass the same safety checks as a typed address are accepted;
- an OPML file can hold at most 2,000 feeds and weigh at most 5 MB.

If the import fails (*The OPML file could not be read.*), open the file in a text editor and look for `<outline type="rss" xmlUrl="...">` lines: some applications use proprietary variants of the format and leave exactly those out.

5.6 Exporting the library to OPML

Exporting lets you keep a copy of the list, move it to another installation or follow the same programmes in a listening app.

1. Open **Settings** → **Archive**, section **Feed list (OPML)**.
2. Press **Export to OPML** and choose a name and location (`feeds.opml` by default).

Any application that supports the standard can read the file produced.

5.7 Exporting an M3U playlist

Export M3U playlist, in the feed header, creates an `.m3u` playlist with the paths of that podcast's archived files, newest to oldest by publication date, and opens the save dialog. The paths point to each file's real location, even when different files sit in different folders.

If there is nothing in the archive yet for that podcast, the program says so (*No archived episodes to put in the playlist.*) instead of creating an empty file.

5.8 Very large feeds

Long-running podcasts and radio archives easily reach thousands of episodes.

RFC 5005 paging. Many platforms put only the recent episodes in the feed and split the rest into pages linked to each other with a `rel="next"` link. The program follows those links by itself and rebuilds the catalogue into a single list. Each page goes through the same safety checks as an address typed by hand, and a broken page does not lose the ones already read. The number of pages followed stops at twenty: a safety cap against feeds that send each other round in circles, which on huge catalogues means the oldest part may be left out.

Analysis takes longer. Two thousand episodes, or a feed split into many pages, can take fifteen to thirty seconds. That is normal.

The list stays light. Even with thousands of entries, only the rows visible on screen are drawn.

Space. Two thousand episodes of fifty-odd megabytes come to around a hundred gigabytes: the **What will be downloaded** panel and the confirmation window show the estimate before you start.

5.9 Checking feeds

Checking a feed means reading it from the server again and comparing its episodes with those already seen. The program does it on four occasions:

- **at launch**, a few seconds after opening;
- **at regular intervals**, according to **Settings → General → Automatic feed checks**: every 6 hours (default), every 12 or every 24, or **Never (manual only)**. The count starts at program launch, or when you change the interval, and the change applies at once;
- **when the connection returns**, after an interruption;
- **on request**, with **Check feeds** (top bar, Library view, command palette or **Ctrl+S**).

Checks carry on even with the window closed to the notification area. The program reads five feeds at a time and, where the server allows it, asks only whether the feed has changed: an unchanged feed is not downloaded again.

How a new episode is recognised. By its GUID, not by counting: an episode is new only if its identifier has never been seen before. This works even with “rolling window” podcasts that drop old episodes as

they publish new ones. The first time the program checks a feed with no reference, for instance one just imported from OPML, it records the episodes present without announcing them as new.

What happens next. The check **downloads nothing**, except for feeds with automatic archiving (section 5.10). It just updates the counts and reports what is new:

- a check started by hand answers with an in-app notice, for instance *5 new episodes found. 2 being archived automatically.* or *14 feeds checked: no new episodes.*, with the number of unreachable feeds if any;
- a background check sends a system notification, in the interface language (*5 new episodes found in 2 podcasts.*). The notification is clickable: it brings the program to the front and, if the news concerns a single podcast, opens that feed directly.

Addresses that change. When a feed answers with a permanent move (a 301 or 308 redirect), the program updates the saved address and carries the history across, so later checks go straight to the new location.

5.10 Automatic archiving per feed

This is how you follow a podcast without lifting a finger: with the switch on, new episodes found by the checks download by themselves.

Turning it on

- When adding the feed, with the **Automatic archiving** switch in the **Add a feed** preview.

- Later, with the **Automatic archiving** switch in the right-hand panel of the feed view (in the action bar, in windows narrower than 1280 pixels).

If the feed already has published episodes that are not archived, the program asks what to do with them when you switch it on, in the *Turn on automatic archiving?* window:

- **Turn on and download the N missing**: switches automatic archiving on and prepares the download of the missing episodes, with the usual confirmation of space and folder;
- **New ones only**: switches automatic archiving on and leaves the back catalogue alone.

Feeds with automatic archiving on carry the **auto** mark in the sidebar and in the Library, and the **Automatic archiving** badge in the header.

How it works

Whenever a check (at launch, periodic, when the network returns or with **Check feeds**) finds new episodes in that feed, it queues them without asking. The same happens when you open or refresh the feed by hand: new episodes are not “used up” by reading the feed, they are queued.

The system notification and the in-app notice report how many episodes went to automatic archiving. The downloads follow the usual rules: parallel downloads, bandwidth limit, automatic pause when offline.

Turning it off

Just switch it off. Episodes already queued stay queued; from then on, new episodes are only reported again.

5.11 Working with many feeds

There is no limit to the number of podcasts in the library, and they all stay available from one session to the next.

- To find one: **Search feeds…** in the sidebar, or the command palette (**Ctrl+K**), which filters feeds by title.
- For the big picture: the **Library** view, sortable by column. Sorting by **To download** puts the feeds with the biggest backlog at the top; sorting by **Last checked** brings out the ones that have not answered for a while.
- To refresh them all: **Check feeds**.

At launch no feed is open: the program shows the Library and waits.

Chapter 6 explains how the download engine works.

Chapter 6: The download engine

6.1 How it is built

The engine works asynchronously and in parallel: instead of downloading one file at a time, it handles several transfers together through a central queue.

There are four parts. The **queue** collects every waiting episode in order, and is saved in the database until each download ends. The **download workers** each take one episode from the queue and see it through, knowing nothing about the others; how many work together is set by a preference. The **database manager** records the state of every transfer. The **integrity check** verifies the size of each newly arrived file, computes its SHA-256 fingerprint and writes it to the archive.

Every way in goes through the same queue: the Feed view's main button, a single episode's download, automatic archiving, resuming interrupted downloads and **Retry**.

6.2 How many parallel downloads

This is the setting to tune first. Too few and archiving drags on; too many and the connection saturates, the server gets overloaded or errors multiply.

The default is **3**, a good compromise on a home connection.

Situation	Parallel downloads
Slow connection, or a server that limits connections	1–2
Normal home connection	3 (default)
Fast fibre, robust servers	5–6
Very fast fibre, many podcasts on different servers	up to 10
NAS reached over a slow network	1–2

Change it from **Settings → Downloads → Parallel downloads**, choosing a value from **1** to **10**. The change applies at once, even to the queue in progress.

If the server complains. Many hosting services limit simultaneous connections per IP address. When errors such as *The server refused the request.* (code `HTTP_429`) or *The server had a problem: try again later.* (`HTTP_503`) keep appearing, drop to one or two. The retry mechanism handles the failures anyway, but reducing the load solves the problem at its source.

6.3 Errors and retries

Across hundreds of files, some network errors are certain. The program does not insist straight away: it waits, and the wait doubles with each failure. Every episode gets **five attempts**.

Attempt	What happens if it fails
First	waits 1 second and tries again
Second	waits 2 seconds and tries again
Third	waits 4 seconds and tries again
Fourth	waits 8 seconds and tries again
Fifth	the episode moves to the **failed** ones

If the server answers `429 Too Many Requests` with a waiting time in the `Retry-After` header, the program honours it, up to sixty seconds. Since the partial `.part` file survives between attempts, the retry resumes from where it stopped rather than from scratch.

The initial connection has thirty seconds to answer; beyond that, the attempt counts as failed (*The server didn't answer in time.*).

Some errors do not deserve a second attempt, and the program recognises them:

- **File not found** (`404`): retrying will not make it appear. Message: *The file no longer exists on the server (404).*
- **Content that is not audio**: the server sent a web page instead of the file, typical of expired links that bounce to a placeholder page. Message: *The server answered with a web page instead of audio.*
- **File too large**: beyond the **Largest file size** you set (section 6.9).
- **Disk full** or **folder not writable**.
- **Address rejected by the safety checks** against SSRF attacks.
- **Cancellation** by the user.

A file that arrives with a size different from the one the server declared (*The file received is incomplete.*) is discarded together with its `.part` and downloaded again from scratch, within the five attempts.

Failed episodes appear in the **Downloads** view, **Failed** section, with their reason. **Retry** on a row, or **Retry all (N)**, puts them back in the queue with the same parameters.

6.4 Stall detection

There is something worse than an error: the transfer that neither finishes nor fails. The connection stays technically open, the operating system reports nothing, but the bytes have stopped arriving. Without a defence, that download would hang there, holding a place in the queue.

It happens mostly with overloaded servers that throttle bandwidth after the first bytes, with routing problems along the way and with large files served by limited distribution networks.

So every active transfer is watched. If not a single byte arrives for **sixty seconds**, the program closes the connection, **keeps the `.part` file** and counts it as a failed attempt: the next attempt resumes from where it stopped. If the stall was momentary, it sorts itself out; if it keeps happening until the attempts run out, the episode ends up among the failed ones with the message *The transfer stalled for over a minute.*

6.5 `.part` files and resuming

During the transfer, content goes into a temporary file with the `.part` extension. The final name comes only when the transfer is complete and the size matches what the server declared. So a truncated audio file with the proper extension never appears in the download folder.

Straight afterwards, in this order, the program writes any ID3 tags (chapter 8), computes the SHA-256 fingerprint of the now-final file and records it in the archive together with size, bitrate and sample rate. The fingerprint therefore describes the file exactly as it is on disk.

Resuming. After a pause, a temporary error or an interruption, a small `.part.meta` file stays next to the `.part`, holding the "validator" the server provided (ETag or Last-Modified). On the next attempt the program asks only for the missing bytes, with an HTTP `Range` request accompanied by `If-Range`: if the remote file has changed in the meantime, the server says so and the download starts again from zero. This guards against the worst risk of resuming — splicing together pieces of two different files. If the server provides no validator, a resume cannot be verified and the download starts again from zero.

Cleaning up. Orphaned `.part` files left by past sessions are removed from **Settings → Archive → Clean temporary files**. It works only with the queue empty and searches every folder the archive uses.

Partial files sit in the same folder as the completed ones. There is no point opening them in an audio player: they are fragments, and the player will reject them.

6.6 Pausing, resuming, stopping

Pause. From the **Downloads** view you can suspend a single transfer, with the pause button on its row, or the whole queue, with **Pause** in the header. The top bar shows **Queue paused**. Pausing keeps the partial files and the place in the queue; **Resume** carries the transfer on from the exact point where it stopped.

Cancelling a download. The × on a queue row, or next to a queued episode in the feed view, takes that episode out of the queue and deletes its partial file. The episode goes back to the ones to download.

Stopping. Stop and clear the queue is something else: after the *Stop every download?* confirmation it closes active transfers, empties the queue and deletes partial files. Whatever was already complete stays in the archive; the interrupted episodes go back to the ones to download. A cleared queue is not offered again at the next launch.

Closing the window. With the × the program retreats to the notification area and keeps downloading. The **Quit** entry of the icon's menu, on the other hand, closes everything and stops transfers: the `.part` files stay on disk and the queue stays saved, so on restart the downloads pick up where they were (section 6.8).

6.7 Automatic pause when offline

When the connection drops, the program pauses the queue by itself. The top bar shows **Offline**, and a notice appears at the top of the main area: *You are offline: the queue is paused and restarts on its own when the connection is back.* Partial files stay on disk.

When the network returns, the queue restarts by itself, from where it stopped, and a few seconds later the program checks the feeds for episodes published in the meantime.

Without this pause, a network drop would turn a batch into a shower of errors in under a minute, because every episode would burn its attempts for nothing.

A pause you chose takes precedence: if you had already paused the queue by hand, the network returning does not restart it.

6.8 A queue that survives closing

Every download accepted into the queue is recorded in the database and removed only when it ends, successfully or not. If the program closes with downloads still queued — quitting with **Quit**, the computer shutting down or a sudden crash — on restart the **Downloads** view shows the *Downloads interrupted when the app last closed* panel, with the number of downloads and the podcasts they belong to.

- **Resume** puts them back in the queue; thanks to the `.part` files, half-finished transfers pick up where they stopped.
- **Discard** forgets them. The partial files stay on disk until temporary files are cleaned.

The program never resumes the queue by itself: the network, the folder or your intentions may have changed in the meantime. Episodes that turn out to be archived already are not offered again.

6.9 Bandwidth limit and largest file size

Two settings in **Settings** → **Downloads**, section **Transfer**, govern how much the engine may take.

Bandwidth limit (KB/s). A single ceiling, **shared by all downloads together**: with a 500 KB/s limit, three parallel downloads share 500 KB/s rather than using 500 each. `0` means no limit and is the default. To give an idea, `500` keeps usage around 4 Mbps. The change applies at once, even to running downloads.

Largest file size (MB). Refuses files bigger than the value given; `0` means no limit. It guards against anomalies: sizes declared by mistake, or servers that send something else entirely instead of the audio. When the server declares the size, the refusal comes before a single byte is written; when it does not, the transfer is stopped as soon as it passes the ceiling. A file over the limit is a permanent error, with no further attempts: *The file exceeds the size limit you set.*

6.10 Speed

Each queue row shows the speed of its own transfer and the time left. The bar under the Downloads view header shows the run's progress in number of files instead.

Four factors decide the speed.

- **Your line**, which remains the ceiling.

- **The origin server**, often the real bottleneck: many hosting services cap bandwidth to keep costs down, and a single transfer rarely goes beyond 2–5 MB/s.
- **The number of parallel downloads**, which makes up for a slow server by using several connections at once.
- **File size**: 20–80 MB files, that is half-hour or one-hour episodes, are the most efficient, because the time spent opening the connection weighs little on the total.

6.11 Disk space

Before queuing several episodes, the program checks the free space on the download folder's disk and compares it with the estimate (chapter 4, section 4.7):

- below 200 MB free, it starts nothing;
- if the free space is less than one and a fifth times the estimate, the confirmation says so in yellow;
- during the batch, a file that no longer fits stops with *Disk full.* and no further attempts, and the others carry on while there is room.

Once space is freed, **Retry all** puts the stopped episodes back in the queue.

Chapter 7 deals with NAS and network-drive paths.

Chapter 7: NAS, network drives and SMB paths

7.1 Why network drives are a special case

Almost every desktop application copes fine with `C:\` and `D:\` and turns unpredictable when the download folder is a NAS, a Windows shared folder or an SMB drive. The reason is technical: a network drive is less reliable than a local one. The NAS may be off or in standby, the network may have a latency spike, credentials may have expired. And an operation on a path that does not answer blocks the program for tens of seconds, freezing the interface.

FeedDownloader Pro is built not to get stuck. This chapter is for anyone keeping the archive on a NAS or a network folder.

7.2 How the path is checked

When you choose a folder with **Settings → Downloads → Change folder**, the program checks that it exists and is writable. The check runs outside the process that draws the interface and has an **eight-second limit**: once that passes, the path is declared unavailable and the warning *The network folder is not answering.* appears. The window never freezes

waiting. If the folder answers but refuses writes, the warning is *The chosen folder is not writable.*

It is a single check, not a battery of tests: the program asks the operating system whether it can write there and takes the answer. No test file is created in the folder.

The program recognises as network paths those starting with `\\` or `//` (Windows UNC and SMB notation) and, on Linux, those under `/mnt/` and `/media/`. When you choose a path of this kind, a notice reminds you that *if the share disconnects, running downloads stop and resume later*. A drive mapped to a letter, for instance `Z:`, is treated by the system as an ordinary disk: the writability check still happens, but the program does not recognise it as a network drive and shows no notice.

Why eight seconds. Home NAS units (Synology, QNAP, WD MyCloud) take three to six seconds to wake from standby. Eight is enough to wait for them and still short enough not to feel like a freeze.

7.3 Setting a NAS path

Direct UNC path. In the form `\\ServerName\ShareName\Folder`:

```
\\MYNAS\Podcasts\Archive
\\192.168.1.100\media\podcast
\\NAS-SYNOLOGY\video\audio_archive
```

Reach it with the **Change folder** picker, which on Windows can browse network locations too; alternatively you can type the path in the picker's address bar.

Mapped drive. If the NAS is already mounted as a drive (for instance `Z:` pointing to `\\MYNAS\Podcasts`), just choose `Z:\Archive`. It works perfectly well; the program simply treats it as a local disk, because at that point the operating system is handling the network.

Linux. Mounted SMB shares appear as ordinary folders (for instance `/mnt/nas/podcast`) and can be used directly.

Moving an existing archive to the NAS. If the archive already holds episodes, when you choose the new folder the program asks what to do with them: **Move the archive** transfers the podcast folders to the NAS, **New downloads only** leaves the files where they are and uses the NAS from then on. Either way the program keeps finding every file (chapter 9, section 9.8).

7.4 Credentials and authentication

NAS credentials are set up in the operating system, not inside FeedDownloader Pro.

Windows. Open the NAS path (`\\MYNAS\`) in File Explorer, enter the credentials when asked and tick the option to remember them. They are stored in Windows Credential Manager (*Control Panel → Credential Manager → Windows Credentials*), and from then on the program reaches the share like any other application.

Linux. Mount the share with credentials in `fstab`, or from a graphical file manager, or by hand with `mount -t cifs`.

7.5 When the network path does not answer

Faced with *The network folder is not answering.*, check in this order.

Is the NAS on? Many home devices go into standby after a period of inactivity. Opening its admin page in a browser is the quickest way to find out.

Can it be reached on the network? From Command Prompt or Terminal:

```
ping 192.168.1.100
```

If it answers, the basic network works and the problem is further up.

Does the share open? Try `\\192.168.1.100\ShareName` directly from the file manager. If it will not open from there, the problem lies in the NAS's SMB configuration, not in the program.

Can you write to it? Create a file in the folder by hand. If the system refuses, the user you connect as has no write permission on that share: fix it from the NAS panel.

Is a firewall in the way? SMB uses port 445, and sometimes 139. Check they are not blocked on the local network.

7.6 Getting good performance

Writing to a NAS means every byte crosses the network twice: speed depends on the local network and on the device's write capacity, not just on the internet connection.

- **Cable beats Wi-Fi.** Wi-Fi adds latency and instability precisely when writing. For large archives, Gigabit Ethernet performs significantly better.
- **Fewer parallel downloads.** Writing many files at once saturates the NAS's I/O: one or two transfers often achieve more than five.
- **Mind the backups.** If the NAS runs automatic backups at fixed times, avoid overlapping download batches: they compete for the same disk and slow each other down.
- **Local first, network later.** For very large archives, download to a fast local disk and then move the archive to the NAS with **Move the archive**.
- **Quick check for everyday use.** Over the network, a full archive check reads back every byte and can take hours; a quick check verifies presence and size in seconds (chapter 9, section 9.5).

Chapter 8 covers file names, templates and metadata.

Chapter 8: File organisation, templates and metadata

8.1 The trouble with names

The name an audio file is published under is rarely meant for human eyes: `ep_2024_03_15_FINAL_v2_mixdown.mp3`, `podcast-episode-187-compressed.m4a`, or even `abc123def456.mp3`. They make sense to the producer's systems; inside an archive they are noise.

The program fixes this with the **naming template**: you define the name format once and every downloaded file follows it, drawing on the information in the feed.

8.2 How a template works

A template is a line of text mixing fixed parts and **placeholders**, variables in curly brackets. At download time each placeholder is replaced with that episode's value.

The template `{date} - {podcast} - {title}` gives:

`2024-03-15 - Mario's Podcast - Episode 187 Artificial intelligence explained.mp3`

The extension is not part of the template: it is added according to the file's format (section 8.6).

8.3 Available placeholders

Placeholder	What it holds	Example
`{title}`	Episode title from the feed	`Episode 187 AI explained`
`{podcast}`	Podcast name	`Mario's Podcast`
`{date}`	Publication date, `YYYY-MM-DD`	`2024-03-15`
`{year}`	Year	`2024`
`{month}`	Two-digit month	`03`
`{day}`	Two-digit day	`15`
`{season}`	Season number, two digits	`02`
`{episode}`	Episode number, at least two digits	`07`, `187`

When an episode declares no date, the date placeholders become the word `unknown`.

`{season}` and `{episode}` are the iTunes numbers declared in the feed (`<itunes:season>` and `<itunes:episode>`). When the feed does not declare them, or puts something other than a whole number there, they become `00`.

A made-up placeholder, for instance `{author}`, stays in the name as it is: the program does not recognise it and leaves it alone.

8.4 Recommended templates

The default is the title alone:

```
{title}
```

Fine for catalogues whose titles are already descriptive.

For general use, put the date first:

```
{date} - {title}
```

which produces `2024-03-15 - Episode 187 AI explained.mp3`. The benefit is practical: alphabetical order of the files matches chronological order.

For podcasts organised in seasons, the numbering gives an even steadier order:

```
S{season}E{episode} - {title}
```

which produces `S02E07 - The return.mp3`. Check first that the feed really declares the numbers: otherwise every file will start with `S00E00`.

If several podcasts end up in the same folder, after a manual move for instance, include the programme's name:

```
{podcast} - {date} - {title}
```

8.5 How names are cleaned up

Some characters are not allowed in file names: on Windows `/`, `\`, `:`, `*`, `?`, `"`, `<`, `>`, `|`, as well as control characters. The program **removes** them from the name produced by the template, without replacing them: `Episode 12: AI` becomes `Episode 12 AI`. The same goes for the name of the podcast's subfolder.

Less obvious cases are handled too:

- **Reserved names.** Windows reserved names (`CON`, `PRN`, `AUX` and the like) cannot be used as file names: if the title boils down to one of them, the program falls back on `episode`. The same happens with a title that ends up empty after cleaning.
- **Length.** The full path, folder included, is kept within 250 characters: if needed, the title is shortened, leaving room for the technical suffixes the program adds while it works (`.part.meta` and any duplicate-name suffix).
- **Duplicate names.** If the folder already holds a file with the same name, or a metadata file with the same base name, the new file gets the `_2` suffix, then `_3` and so on. Two episodes with the same title never overwrite each other, not even when they download at the same time.

On long titles. Some podcasts use hundred-and-fifty-character titles. `{title}` on its own gives unwieldy names that risk being cut short; with

`{date}` in front you still get a sortable, recognisable name even when the end gets trimmed.

8.6 The file extension

The program saves every file with its real extension. It takes it from the file's address when the address contains one (`.mp3`, `.m4a`, `.m4b`, `.aac`, `.ogg`, `.opus`, `.flac`, `.wav`, `.mp4` and other audio and video formats); when the address does not say, it uses the MIME type declared in the feed, so `audio/mp4`, for instance, becomes `.m4a`. Only when both pieces of information are missing does the file become `.mp3`.

8.7 Where to set it

The template is set in **Settings → Files and metadata**, section **File names**:

1. Type the template in the **Template** field, or build it with the **Available placeholders** buttons: each click adds the placeholder at the end of the template.
2. Check the **Example** under the field, which shows the result on a sample episode.
3. Leave the field: the template is saved. A field left empty goes back to `{title}`.

The template applies to future downloads: **existing files are not renamed**. It can be at most 300 characters long.

8.8 One thing the template does not do

The template produces **the file name, not a folder structure**. Writing `{year}/{month}/{title}` does not create year and month subfolders: the slashes are removed along with the other forbidden characters, and the result is a single file called `202403Title.mp3`.

The program applies only one automatic organisation, and it concerns podcasts: inside the download folder each programme has its own subfolder, named after the podcast.

8.9 Metadata next to the files

In **Settings** → **Files and metadata**, section **Metadata**, there is the **Save metadata next to the files** switch, off by default. When it is on, the program writes three kinds of file next to the audio files.

A `.json` file for each episode

It has the same name as the audio file, with a `.json` extension, and records everything known about the episode:

```
{
  "schema": "feeddownloader.episode/1",
  "title": "Episode 187: AI explained",
  "podcast": "Mario's Podcast",
  "feedUrl": "https://www.example.com/feed.xml",
  "guid": "https://www.example.com/ep187",
  "pubDate": "Fri, 15 Mar 2024 08:00:00 GMT",
  "link": "https://www.example.com/episodes/187",
  "description": "In this episode…",
  "duration": "00:52:14",
  "season": "2",
  "episode": "187",
  "episodeType": "full",
  "enclosure": {
    "url": "https://media.example.com/ep187.mp3",
    "type": "audio/mpeg",
    "length": 75210432
  },
  "file": {
    "name": "2024-03-15 - Episode 187 AI explained.mp3",
    "size": 75231890,
    "sha256": "9f2c…",
    "bitrateKbps": 192,
    "sampleRateHz": 44100
  },
  "downloadedAt": "2026-10-23T09:14:22.517Z",
  "savedBy": "Runtime FeedDownloader Pro 1.6.0",
  "sourceUrl": "https://media.example.com/ep187.mp3",
  "filename": "2024-03-15 - Episode 187 AI explained.mp3"
}
```

Date, duration and notes are recorded as the feed publishes them, so the notes may contain HTML. The `sourceUrl` and `filename` fields repeat information found elsewhere and remain for scripts written for earlier versions.

A `podcast.json` for each podcast

In the podcast's folder, with the `feeddownloader.podcast/1` schema: title, feed address, website, description, author, language, copyright, categories, cover address, and the date and program version that wrote it. It is refreshed once per session, at the podcast's first download.

The cover

In the same folder, `cover.jpg` (or `.png`, `.webp`, `.gif`, depending on the original format): the podcast image declared in the feed. It is downloaded only if the folder does not already hold a cover.

What they are for

Two things: letting scripts and outside systems read the metadata without going through the database, and keeping the archive readable on the day the feed disappears. With these files a podcast folder is self-sufficient: audio, notes, numbering, fingerprints, description and cover travel together.

8.10 ID3 tags

Also in **Settings → Files and metadata**, the **Write ID3 tags into MP3s** switch, off by default, writes information inside the file:

- **Title**: the episode title;
- **Artist** and **Album**: the podcast name;
- **Year**: the year of publication;

- **Cover**: the episode image if the feed declares one, otherwise the podcast's, but **only if the file has no embedded cover already**.

Writing is **non-destructive**: tags already in the file, such as the publisher's chapters, comments, links and cover, are kept. All the common audio players read the tags, so the information stays visible even if the file name changes.

Tags are written **before** the SHA-256 fingerprint is computed, so the recorded fingerprint describes the file exactly as it is on disk. Versions up to 1.5.x did the opposite: if you used them with tags on, section 9.5 explains how to put the fingerprints right.

MP3 only. `.m4a`, `.ogg`, `.opus` and other formats are not modified, even with the option on.

Chapter 9 covers file integrity, verification and archive management.

Chapter 9: Integrity, verification and archiving

9.1 Why check the files

A download that reaches the end is not necessarily a sound file. A packet lost on the way, a write error, an interruption in the last second: the file is there, it weighs nearly what it should, and it turns out to be incomplete on the day someone tries to listen to it. For an archive that is the worst way to go wrong, because the damage shows up years later.

FeedDownloader Pro works on two levels: it checks the **size** while downloading and computes the **SHA-256 fingerprint** as soon as the file is complete. Then, over time, **archive verification** compares the disk with what was recorded.

9.2 The SHA-256 fingerprint

SHA-256 produces, for any file, a sequence of 64 hexadecimal characters. Two identical files give the same fingerprint; change a single bit and the fingerprint becomes unrecognisable. It is the standard way of saying “this file is exactly the one from back then”.

For each download the program computes the fingerprint and records it in the archive together with the file name, the folder and the date. The computation happens **after** any ID3 tags are written, so on the file exactly as it is on disk.

It serves three purposes. The **full check** (section 9.5) recomputes the fingerprint of the files present and compares it with the recorded one, uncovering corruption and changes made after the download. **Find moved files** uses the same fingerprint to recognise files someone renamed or moved by hand. And in a professional setting the fingerprint is verifiable proof of what that file contained when it was archived.

You can read the fingerprint in the episode details (**SHA-256 fingerprint (recorded at download)**), in the CSV inventory and, if enabled, in the metadata next to the files.

9.3 Technical metadata

Once a download finishes, the program reads the technical data **inside the file**, not from the feed, and records it.

Field	What it tells you	Example
Bitrate	Audio quality in kilobits per second	`128 kbps`, `320 kbps`
Sample rate	Samples per second	`44.1 kHz`, `48 kHz`
Size on disk	The file's real weight	`67.4 MB`

They appear in the episode details (**Quality** and **Size on disk**), in the Archive view (the **Size** and **Quality** columns) and in the CSV inventory.

9.4 The Archive view

The **Archive** view (**Ctrl+3**) brings every downloaded episode, from any podcast, into a single table, with publication and download dates, size and quality. Its layout is described in chapter 3, section 3.9.

Some typical uses:

- **What do I have of a podcast?** Click the podcast in the left-hand column: the summary line says how many episodes and how much space.
- **What came out in a period?** The **Publication period**, with a ready-made range or the **From** and **To** dates.
- **Where is a file?** Search also accepts the file name; the folder icon shows it in the file manager, wherever it is.
- **What arrived today?** The default order, by download date, newest first.

9.5 Archive verification

Over time an archive changes behind the program's back: files moved, renamed, deleted by mistake, retouched by another program, or ruined by a failing disk. Verification compares what is on disk with what the archive records.

Starting it

The **Archive verification** window opens in three ways:

- **Verify archive** in the Archive view header;
- **Verify archive** in **Settings** → **Archive**;
- **Verify archive** in the command palette (**Ctrl+K**).

At the top choose the **Kind of check**, then press **Start check**.

Quick or Full

Kind	What it checks	Duration
Quick	That every file exists where expected and has the recorded size	A few seconds, even on large archives
Full	In addition, recomputes each file's SHA-256 fingerprint and compares it with the recorded one	Proportional to the gigabytes: on large archives or a NAS it can take hours

The quick check suits frequent use: it finds missing, truncated or replaced files. The full check also finds damaged files that kept their size, and is the one to run periodically on an archive that matters. Rows recorded by older versions, with no fingerprint, are checked for presence only.

During the check a bar shows progress (*Checking…* with the number of files checked out of the total) and the **Stop the check** button interrupts it. A stopped check reports the results for the files already examined (*Check stopped: the results cover the first N episodes.*). While it runs, the window cannot be closed.

The program looks for each file in the folder it was downloaded to; if it is not there, it also looks in the current download folder.

The results

At the end five figures appear:

Indicator	Meaning
Checked	Episodes examined
Present	Files found
Missing	Files not found
Different size (quick) or **Different from original** (full)	Files present but no longer identical to the ones recorded
On disk	Total space of the files present

If there is nothing to report, *All good: every file is present and intact.* appears. Otherwise one or two sections appear, each with the list of files involved (the first fifty, with a count of the rest).

Missing files

These are files moved, renamed or deleted. Two actions, to use in this order:

1. **Find moved files.** Searches the podcast's folder for files someone renamed or moved by hand, both in the folder the episode was downloaded to and in the current download folder. It recognises them by their SHA-256 fingerprint: for each missing episode it computes the fingerprint of candidate files of the same size and,

when it finds the right one, relinks the file to the episode under its new name and location. It downloads nothing and touches no files: it updates the archive. At the end it says how many files it found and how many it did not (*3 files found and relinked · 1 not found.*) and runs the check again. It works only for episodes with a recorded fingerprint.

2. **Mark the missing ones as not downloaded.** This is for files that are truly lost: after a confirmation it removes them from the archive, so the episodes go back to **To download** and can be downloaded again as usual.

Files different from the original

These are files present but with a size or fingerprint different from the recorded one. There are two causes:

- **damage**: a disk error, an interrupted copy, a file replaced by mistake. In this case the file should be downloaded again: open the episode and press **Download again**;
- **a deliberate change**: tags written by another program, a trimmed or normalised file. In this case the file is fine; it is the reference that changed.

For the second case there is **Accept the current files as reference**: it records the fingerprint and size of the files now on disk as the new reference, and the check stops reporting them. Use it only after checking that the files play correctly.

Files tagged by versions up to 1.5.x. Those versions computed the fingerprint before writing ID3 tags, so every tagged MP3 shows as **Different from original** on the first full check with 1.6.0. This is exactly

the case for **Accept the current files as reference**: after spot-checking that they play, one click puts all the fingerprints right.

The check at launch

A few seconds after every launch, the program checks in the background that the archived files still exist. If some are missing, it sends a system notification (N *archived files are missing from disk…*) inviting you to open the verification. The check changes nothing.

9.6 Statistics

The summary figures appear in three places:

- **Settings → Archive**: N *episodes from M podcasts since [date of the first download].*;
- **the Archive view**: **Episodes**, **Podcasts** and **Space used**, plus the range of publication dates;
- **the Library view**: **Feeds**, **To download** and **Archived**, and for each feed the space used.

They update by themselves after every completed download.

9.7 The CSV inventory

Exporting produces a file with one row per episode: for spreadsheets, editorial systems, scripts, or simply to have a list of what you own outside the program.

Create it with **Export CSV**, in the Archive view header or in **Settings → Archive**, choosing where to save it (`archive_report.csv` by default).

Column	Contents
`Podcast`	Podcast name
`Episode Title`	Episode title
`Publish Date`	Publication date
`Downloaded At`	Download date and time
`File Size (bytes)`	Size in bytes
`Bitrate (kbps)`	Audio bitrate
`Sample Rate (Hz)`	Sample rate
`SHA-256 Checksum`	File fingerprint
`Validation Status`	See below
`GUID`	The episode's identifier in the feed
`Feed URL`	Feed address
`File`	File name

Column	Contents
`Folder`	The download folder that holds the podcast's subfolder

The last three columns are new in 1.6.0 and sit at the end, so spreadsheets that read the first ten keep working.

The `Validation Status` column says what level of assurance the episode was recorded with, not the result of the latest check: `OK` when a SHA-256 fingerprint is recorded, `LEGACY` for older rows that have the file name but no fingerprint, `UNKNOWN` when even that is missing.

Format. Comma-separated CSV, UTF-8 with a BOM for Excel compatibility, every field in quotes. Values that would start with `=`, `+`, `-` or `@` get a leading apostrophe, so a spreadsheet does not run them as formulas.

9.8 Changing the download folder

Every archived episode remembers the folder it was saved in. Changing the download folder therefore does not make the archive you have already built go "missing": the program keeps finding every file.

Change the folder from **Settings → Downloads → Change folder**. If the archive is empty, the new folder applies at once. If it already holds episodes, the program asks *What about the existing archive?* and offers two routes.

Move the archive

Moves the subfolders of the current download folder into the new one, updates the archive and uses the new folder for future downloads.

- During the move, Settings shows progress (*Moving: 3 of 12 folders*); at the end the program says how many folders it moved (*Archive moved: 12 folders.*).
- Each podcast remembers where it really is: if some folders cannot be moved, for instance because a folder with the same name already exists at the destination, they stay where they were and the program keeps finding them (*10 folders moved, 2 not moved: …*). If none can be moved, the download folder does not change.
- The move does not start while downloads are queued (*Wait for the downloads to finish before moving the archive.*), nor when the new folder is inside the current one or contains it.

Note: this is a move, not a copy: the files leave their original location. Check first that the destination has room.

New downloads only

Leaves the files where they are and uses the new folder for future downloads only. The archive ends up split across two (or more) folders, but the program knows where every episode is: the Archive view, **Show in folder**, M3U playlists, verification and the search for moved files all work across them.

This is the right choice when a disk fills up: the podcasts already archived stay on the old disk, new episodes go to the new one.

9.9 Moving to another computer

You need two things: the folder (or folders) of audio files and the `feeddownloader.sqlite` database (chapter 2, section 2.4).

1. Install the program on the new computer and close it.
2. Copy the database into the data folder, replacing the one created at installation.
3. Copy the audio files. If they end up at the same path as before, nothing else is needed.
4. If they end up at a different path, open the program and set that path with **Change folder**, choosing **New downloads only**. The program looks for each file first in the recorded folder and then in the current download folder, so it finds them, as long as each podcast keeps its own subfolder. A quick check confirms it; for files that have been renamed in the meantime there is **Find moved files**.

Chapter 10 goes through the settings.

Chapter 10: Settings

10.1 How they are organised

The panel opens at any time from the gear at the top right, with **Ctrl+,** or from the command palette, even while a download is running. The options are split across six vertical tabs, in this order: **General**, **Downloads**, **Files and metadata**, **Archive**, **Accessibility**, **About**.

Every change is saved at once: switches and menus when you choose, number fields and the naming template when you leave the field. The **Done** button closes the panel.

This chapter goes through the tabs one by one. The naming template, the metadata next to the files and ID3 tags are covered in chapter 8; archive verification, the CSV inventory and changing the folder in chapter 9.

10.2 General

Interface language

The interface speaks **Italiano** and **English**. The choice also applies to system notifications and changes everything instantly, without restarting. The same choice is available on the entry screen.

Automatic feed checks

Sets how often the program looks for new episodes in every feed, even with the window closed to the notification area. A check happens anyway at launch and when the connection returns after an interruption.

Interval	Behaviour
Never (manual only)	No periodic checks; the one at launch and **Check feeds** remain.
Every 6 hours (default)	All feeds read again every six hours.
Every 12 hours	Read again every twelve hours.
Every 24 hours	Read again once a day.

The change applies at once. When a check finds new episodes, feeds with automatic archiving download them straight away; for the others a clickable system notification arrives (chapter 5, section 5.9).

10.3 Downloads

Download folder

Shows the current path; by default `Documents\FeedDownloader\downloads`. Each podcast gets its own subfolder, and an external disk or a network folder (NAS, SMB) works too.

Change folder opens the system picker, checks that the folder is writable (with an eight-second limit for network paths, chapter 7) and, if the archive already holds episodes, asks whether to **Move the archive** or use the new folder for **New downloads only** (chapter 9, section 9.8). During a move the button stays disabled and progress appears under the path.

Transfer

A line recalls the engine's fixed behaviour: each download retries by itself up to five times on transient errors and resumes partial files from where they stopped. The internal parameters (connection timeout, number of attempts, stall watch) cannot be changed.

- **Parallel downloads**: how many transfers at once, from **1** to **10**. The default is 3. For help choosing, see chapter 6, section 6.2.
- **Bandwidth limit (KB/s)**: the overall ceiling, shared by all downloads together. A reminder under the field repeats it (*Total for all downloads together*: 500 KB/s.). `0` means no limit and is the default.
- **Largest file size (MB)**: bigger files are skipped, with a permanent error. `0` means no limit and is the default.

Details of the limit and the largest size are in chapter 6, section 6.9.

10.4 Files and metadata

- **File names**: the **Template** field, the **Available placeholders** buttons and the **Example** of the result. The template applies to future downloads; existing files are not renamed (chapter 8, sections 8.2–8.7).

- **Metadata**: the **Save metadata next to the files** (chapter 8, section 8.9) and **Write ID3 tags into MP3s** (chapter 8, section 8.10) switches, both off by default.

10.5 Archive

Archive

The summary (*N episodes from M podcasts since …*) and two commands: **Verify archive**, which closes Settings and opens the verification (chapter 9, section 9.5), and **Export CSV** (chapter 9, section 9.7). With the archive empty, both stay disabled.

Feed list (OPML)

Import from OPML and **Export to OPML**, to bring feeds in from, or out to, another podcast application. Importing does not change feeds already in the library (chapter 5, sections 5.5 and 5.6).

Temporary files

Clean temporary files deletes the `.part` and `.part.meta` files left by interrupted downloads that will not be resumed, in every folder the archive uses. It does not work while downloads are queued, because those files may be in use: the button stays disabled (*Wait until the queue is empty.*). At the end it reports how many files it removed.

Reset history

Reset download history empties the archive and the download history — the memory of what has already been fetched: every episode goes back to **To download**. The program asks for confirmation (*Reset the history?*) and the operation cannot be undone. The audio files on disk **stay where they are**, but the program no longer recognises them as downloaded: downloading those episodes again would create copies with the `_2` suffix.

It makes sense in one case only: when you want to start again with a clean archive, for instance at the end of a round of testing. The command stays disabled while downloads are queued. Before using it, copy `feeddownloader.sqlite` somewhere safe.

10.6 Accessibility

Interface size

Enlarges text and controls. The **Size** menu offers 80, 90, 100, 110, 125, 140, 160 and 180%. You can step through the same values from the keyboard with **Ctrl +** and **Ctrl −**; **Ctrl 0** goes back to 100%. The choice is remembered.

With the interface enlarged there is less room available, and the layout switches sooner to its narrow-window variants (chapter 3, section 3.14).

Motion

The **Reduce animations and effects** switch removes transitions and animations, including the entry screen's. If you have already asked the

operating system for reduced motion, animations are reduced anyway, even with the switch off.

The program also respects Windows high-contrast mode. The interface colours meet WCAG 2.2 AA contrast, text never goes below 12 pixels and keyboard focus is always visible.

10.7 About

Updates

Shows the **Installed version** and the update status. The update system works **with consent**: nothing downloads and nothing installs by itself.

At launch, in the installed version, the program queries the project's releases page. If it finds a new version, the **Version X.Y.Z available** notice appears in the top bar along with a system notification, but the package **is not downloaded**. The button changes with the status:

- **Check for updates** forces the check; while it runs *Checking for updates…* appears, then *You have the latest version.* or the version available;
- **Download** starts the download, with *Downloading update* N%;
- **Restart and install**, once the package is ready, closes the program, applies the update and reopens it. Installation does not happen by itself either, and it does not happen when the program closes.

If the check fails, *Updates could not be checked.* appears.

Next to it, **What's new** reopens the release notes of the installed version.

Documentation

Guide opens the built-in documentation; **Full manual (PDF)** opens this manual in the browser, in the interface language.

The project

The licence, the author and the **Contact**, **Source code** and **Support the project** links, which open in the browser.

10.8 How addresses are filtered (for information)

This section explains something the program does by itself and that cannot be configured.

A feed is content that comes from outside, and the addresses in it could point to resources on your local network rather than a public audio file: the router's admin page, a NAS, an internal service. This is the family of attacks called SSRF. An archiver that blindly followed a feed's addresses would become the tool for reaching the network of whoever uses it.

So every address — of the feed, of its pages, of the audio files and of the covers — goes through five filters:

1. **Syntax**: it must be a valid address.
2. **Protocol**: only `http://` and `https://`. Schemes such as `file://`, `ftp://`, `data:` or `javascript:` are rejected at once.
3. **Internal names**: `localhost` and loopback names are refused.

4. **Private and reserved addresses**: the RFC 1918 networks (`10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16`), loopback `127.0.0.0/8` , link-local `169.254.0.0/16` , the CGNAT space, and for IPv6 `::1` , `fc00::/7` and embedded IPv4 addresses.
5. **Check at connection time**: the IP address the name resolved to is checked again on every connection and every redirect. This is the filter that closes the cleverest loopholes — a public domain that resolves to a private address, or a redirect that halfway through points inside the network.

When a filter steps in, the message is *Local-network addresses are not allowed.* for a feed, and *The address points to the local network: blocked for safety.* for a download.

A *stated limitation.* If your network has an internal podcast server reachable at a private address, those filters will block it, and there is no way to make an exception: there is no allow-list to configure. This is deliberate, because a configurable exception is exactly what a hostile feed would try to exploit. Content like that needs another route, for instance downloading it with a different tool.

Chapter 11 collects the most common problems and how to solve them.

Chapter 11: Troubleshooting

11.1 How to use this chapter

Here are the problems that actually happen, described as they appear on screen rather than by their names inside the program. For each one: why it happens and what can be done.

If your case is not listed, the project's public page is the right place to report it, giving the steps that reproduce it and the exact message that appears.

11.2 Feeds and analysis

Analysis error messages

When a feed cannot be read, one of these messages appears under the **Add a feed** field (or in a notice, when you open a feed from the library).

Message	What it means
The address leads to a web page, not to an RSS feed.	The address is the website's, not the feed's: look for the RSS link on the page (chapter 4, section 4.2).

Message	What it means
The feed no longer exists at this address (error 404).	The feed has moved or been removed. Look for the new address on the podcast's website.
The server refused the request: the feed may be private.	The server asks for credentials or blocks access (errors 401 and 403).
The server is not answering. Check the connection.	No network, server down or no answer within fifteen seconds.
The address is not valid.	A typo, extra spaces or characters, or a protocol other than HTTP or HTTPS.
Local-network addresses are not allowed.	The address points to the local network and is blocked by the safety checks (chapter 10, section 10.8).
This feed was just requested: try again in a few seconds.	The same address was requested less than three seconds earlier. Just wait.
The feed could not be read.	Any other case: a malformed document or an unrecognised format.

The likeliest causes, in order:

- **The feed server is not answering.** Try opening the same address in a browser: if it will not open there either, the problem belongs to the podcast and the only move is to try later.
- **Your connection is unstable.** Check that other sites respond.
- **A firewall or corporate proxy is blocking the request.** If the feed opens at home and not at the office, that is the cause.

The feed is read but the list is empty

- **The feed has no episodes.** Open the address in a browser and look for `<item>` or `<entry>` tags. If there are none, there is nothing to download (*This feed has no episodes.*).
- **The filters hide everything.** With *No episode matches the filters.* just press **Clear filters**.
- **The format is non-standard.** The program reads RSS 2.0 and Atom 1.0 and tolerates plenty of sloppiness, but some proprietary platforms produce unrecognisable documents.

The main button is disabled

The **Download the N missing** (or **filtered**) button stays disabled in two cases: there is no network, or nothing among the episodes shown is left to download, because they are all archived or already queued. The **Archived** filter confirms it at a glance.

I only see the latest episodes, not the back catalogue

Why: the limit is set by the publisher, not the program. Many platforms expose only the last 50 or 100 episodes in the feed to spare their own servers.

What the program already does: if the back catalogue is published in linked pages following the RFC 5005 standard, the program follows them by itself and rebuilds the catalogue, up to twenty pages. The problem remains only when there is no paging, or when the catalogue goes beyond that cap.

What you can try: look for an alternative “full feed”, which some platforms offer; fetch the old episodes from the podcast’s website; check

whether the platform accepts parameters in the address such as `?limit=0`.

A feed stays “Not checked yet”

This is the state of feeds imported from OPML or never read. Right after an import the program checks every feed; if one stays in this state, it did not answer. Opening it from the sidebar shows the reason.

11.3 Downloads

Download error messages

In the **Downloads** view, **Failed** section, every failed episode shows a message and, in brackets, the technical code.

Message	Code	What it means
The file no longer exists on the server (404).	`EPISODE_NOT_FOUND`	The publisher has removed the file. No further attempts.
The transfer stalled for over a minute.	`DOWNLOAD_STALLED`	Sixty seconds without data, repeated until the attempts ran out.
The server didn’t answer in time.	`DOWNLOAD_TIMEOUT`	No answer to the connection within thirty seconds.

Message	Code	What it means
Disk full.	`DISK_FULL`	There is no room left in the download folder.
The file received is incomplete.	`INTEGRITY_CHECK_FAILED`	The size received does not match the declared one, on every attempt.
The download folder is not writable.	`PERMISSION_DENIED`	Insufficient permissions, a protected disk, a file locked by another program.
The server answered with a web page instead of audio.	`INVALID_CONTENT_TYPE`	An expired link or a placeholder page in place of the file.
The file exceeds the size limit you set.	`FILE_TOO_LARGE`	The file is beyond the **Largest file size**.
The address points to the local network: blocked for safety.	`URL_PRIVATE_IP` and similar	Blocked by the anti-SSRF checks.
The server refused the request.	`HTTP_4xx`	Access denied, too many requests (`HTTP_429`) or another client error.

Message	Code	What it means
The server had a problem: try again later.	`HTTP_5xx`	An error on the origin server.
Network error.	various	Dropped connection, server name not resolved, connection refused.
Unexpected error.	various	Any other case.

Retry on a row, or **Retry all (N)**, puts the episodes back in the queue with the same parameters.

Many episodes fail with "The file no longer exists on the server (404)."

Why: the episodes are still listed in the feed, but the audio files are gone. It happens a lot with abandoned or migrated programmes.

There is no way to download a file that does not exist. But if the podcast is active and the errors are many, it is worth telling the publisher: sometimes it is a botched migration, and it gets fixed.

Worth knowing: failed episodes are **not excluded** from later batches. They stay **To download** and a new **Download the N missing** will try them again. This is deliberate, because a file can become available again; if they get in the way, a period filter or a manual selection keeps them out.

Downloads are very slow

- **The server caps bandwidth.** The most common case. With servers that penalise multiple connections, dropping to one or two parallel downloads sometimes helps.
- **A bandwidth limit is on. Settings → Downloads → Bandwidth limit (KB/s)** applies to all downloads together: with three parallel downloads and 500 KB/s, each gets about a third.
- **You are on Wi-Fi.** For long batches a cable is steadier.
- **The destination disk is slow**, typically a NAS over Wi-Fi or a USB 2.0 stick. Download locally first and move later with **Move the archive**.
- **That is your line.** A speed test settles it.

An episode sits at 99% and never finishes

Why: the server stopped sending data without closing the connection. Within sixty seconds the stall watch notices, closes the transfer and counts it as a failed attempt; the next attempt resumes from where it stopped thanks to the `.part` file.

If it keeps happening until the five attempts run out, the episode ends up among the failed ones. It usually means the file on the server is truncated or corrupt: you can tell from the message *The file received is incomplete.*

The queue stopped by itself

Three possibilities, which the top bar tells apart:

- **Offline**: the connection dropped and the queue paused by itself. It restarts by itself when the network is back (chapter 6, section 6.7).

- **Queue paused**: the pause was requested by hand. The network returning does not lift it: you need **Resume** in the Downloads view.
- **Queue empty** with some failed: the remaining downloads failed. The reason is in the **Failed** section.

After a restart the queue is gone

It is not gone: it is saved. Open the **Downloads** view and the *Downloads interrupted when the app last closed* panel appears, with **Resume** and **Discard** (chapter 6, section 6.8). The program does not resume it by itself.

A downloaded file is reported as corrupt by the player

Why: it should not happen, given the `.part` files, the size check and the refusal of content that is not audio. If it does, either the file on the server was already damaged, or there was a write error on the disk.

What to do, in order:

1. Open the episode details by clicking its title and press **Download again**.
2. If the file arrives broken a second time, the problem is at the source: check by opening the file address (**Copy audio** in the details) directly in a browser.
3. Run a **Full** archive check (chapter 9, section 9.5): it lists every file whose content no longer matches what was recorded at download.

You need to pause without losing the work done

There is no need to stop everything for good: **Pause**, in the Downloads view, suspends the queue while keeping the partial files, and **Resume** restarts the transfers from the exact point they had reached. It works

for a single episode too. **Stop and clear the queue**, on the other hand, deletes the partials: it is the right choice only when you really want to throw the work away.

11.4 Archive and verification

The full check reports every MP3 as different from the original

Why: those files were downloaded with a version up to 1.5.x with ID3 tags on. Those versions computed the fingerprint before writing the tags, so the recorded fingerprint does not match the final file.

What to do: spot-check that the files play correctly, then press **Accept the current files as reference** in the verification window (chapter 9, section 9.5). With 1.6.0 the problem does not come back.

"The file is no longer where it was saved."

It appears when you press **Show in folder** on an episode whose file is gone: it was moved, renamed or deleted outside the program. The program at least opens the podcast's folder, if it exists. Archive verification, with **Find moved files**, finds files renamed or moved into another of the known folders; the truly lost ones can be marked as not downloaded.

"The folder will be created with the first download."

It appears when you press **Open folder** on a feed with nothing downloaded yet: the podcast's subfolder does not exist yet. It is not an error.

Files are missing after changing folder

With 1.6.0, changing folder does not make the existing archive go missing, because every episode remembers its own folder (chapter 9, section 9.8). If files are missing, they were moved by hand to a place the program does not know. Two remedies:

- set the folder they are in now as the download folder, with **New downloads only**: the program also looks there;
- or put them back in their original place.

A quick check confirms the result.

11.5 NAS and network

"The network folder is not answering." but the NAS works

1. **Check the path character by character.** On some systems case matters too (`\\MYNAS\podcast` is not `\\MYNAS\Podcast`).
2. **Check the credentials.** Open `\\MYNAS\ShareName` in File Explorer: if it asks for the password, it is not stored in Windows Credential Manager. Enter it and tick the option to remember it.
3. **Check the firewall.** In *Windows Defender Firewall → Allowed apps* the program must be authorised.

4. **Check the SMB version.** Older NAS units only speak SMBv1, which Windows 11 disables by default. Updating the NAS firmware is better than re-enabling an insecure protocol.

Downloads to the NAS stop after a few minutes

Why: the NAS goes into standby while working. Some home models only watch web traffic and ignore SMB connections, so they think they are idle while receiving files.

Fix it by temporarily turning off power saving in the NAS panel, or by dropping to a single parallel download: a continuous stream keeps the device awake better than bursts separated by pauses.

11.6 General problems

The interface is too small, too large or shows tabs instead of panels

Adjust the interface size in **Settings** → **Accessibility** → **Size**, or with **Ctrl** **+**, **Ctrl** **−** and **Ctrl** **0**. With the interface heavily enlarged, or in a narrow window, side panels become tabs or windows (chapter 3, section 3.14): widen the window, or reduce the size, and they sit side by side again.

The interface responds slowly

- **A very large archive.** With tens of thousands of recorded episodes some operations slow down.
- **Too many parallel downloads on little memory.** Many transfers on a machine with less than 4 GB are noticeable: three or fewer is better.

- **The antivirus inspects every `.part`.** Many products scan every write to disk. Excluding the download folder from real-time scanning fixes it.

The notification-area icon is missing

On Linux with GNOME you need the AppIndicator extension. Without a notification area the program cannot shrink to an icon, and closing the window quits it: the queue is still saved and resumes on restart.

The program will not start, or closes at once

1. **Damaged database: automatic recovery.** If `feeddownloader.sqlite` is unreadable at launch, the program sets it aside, renaming it `feeddownloader.sqlite.corrupt-[date]`, and starts with a new database. The application opens regardless.
2. **Guided restore.** On the first launch after a recovery — with the database empty and a `.corrupt-*` backup present — the *A backup of a damaged database was found.* window appears, with **Attempt restore** and **Ignore**. The restore recovers feeds, archive and history from the backup as far as they are readable: audio files are not touched and the backup is not modified. At the end a summary says how much was saved.
3. **Reinstall.** Uninstalling and reinstalling the latest version deletes neither the database nor the settings.

The database is lost: can anything be recovered?

- **With a backup**, just put `feeddownloader.sqlite` back in the data folder with the program closed (the path is in chapter 2, section 2.2).
- **If the database became corrupt**, look for `feeddownloader.sqlite.corrupt-[date]` files in the same folder: with

the current database empty, the program offers the guided restore by itself at launch.

- **With neither**, the audio files are still safe on disk: the memory is lost, not the archive. Start again by adding the feeds; the metadata next to the files, if it was on, keeps every episode's title, date and fingerprint. The search for moved files does not help here, because the reference fingerprints were in the very database that was lost.
- **To avoid it**, copy `feeddownloader.sqlite` elsewhere from time to time and export the feed list to OPML (chapter 5, section 5.6). Before a migration or a major update it is time well spent.

This is the end of the Runtime FeedDownloader Pro manual. For anything these pages do not cover, the reference is the project's public page.